

Agent Security is a Systems Problem

Mihai Christodorescu¹ Earlence Fernandes² Ashish Hooda¹ Somesh Jha^{1,3}
Johann Rehberger⁴ Kamalika Chaudhuri² Xiaohan Fu^{2,5} Khawaja Shams¹
Guy Amir⁶ Jihye Choi³ Sarthak Choudhary³ Nils Palumbo³
Andrey Labunets² Nishit V. Pandya²

¹Google ²University of California San Diego ³University of Wisconsin–Madison
⁴EmbraceTheRed ⁵Gray Swan AI ⁶Cornell University

Abstract

We take the position that agent security must be approached as a systems problem: the AI model powering the agent must be treated as an untrusted component, and security invariants must be enforced at the system level. Through this lens, efforts to increase model robustness (the dominant viewpoint in the community) are insufficient on their own. Instead, we must complement existing efforts with techniques from the systems security domain. Based on our experience as cybersecurity researchers in operating systems, networks, formal methods, and adversarial machine learning, we articulate a set of core principles, grounded in decades of systems security research, that provide a foundation for designing agentic systems with predictable guarantees. As evidence, we analyze eleven representative real-world attacks on agents and discuss how systems principles, if realized, could have prevented these attacks. We also identify the research challenges that stand in the way of implementing these principles in agents.

1 Introduction

Position. Agent security must be approached as a systems problem: the AI model powering the agent must be treated as an untrusted component, and security invariants must be enforced at the encompassing system level. Building upon security principles grounded in decades of research, one can identify the three main gaps that need to be bridged to improve the secure deployment of agentic systems in the wild.

Agentic computing is an emerging paradigm where AI models employ various *tools* to complete tasks that users specify in natural language. These systems operate in complex environments, such as websites, applications, and external services, and are exposed to adversarial inputs which can influence their behavior. We consider a threat model where an attacker controls some portion of the agent’s input channels (e.g., documents, tool outputs, etc.) with the goal of inducing unintended actions or exfiltrating sensitive data. For example, an agent may be prompt injected by malicious external content (e.g., a webpage) and coerced into misusing its access to user data [27, 44, 46, 48, 49, 51–53, 64, 69]. More broadly, the AI model powering the agent itself is an unreliable component that may behave unpredictably under adversarial or ambiguous inputs.

A growing body of work has emerged to address these risks. These approaches treat the model as the primary object of security and aim to make it *directly* robust to a wide range of adversarial inputs, using alignment techniques. We argue that this approach is fundamentally limited: (1) it was attempted in the days of “classical adversarial ML” that dealt with computer vision models and it was repeatedly shown that attackers can evade model-based defenses [5]; (2) a similar phenomenon has been observed in the current LLM era, where attackers always appear to be successful, without a substantial increase in attacker effort [36, 42].

Furthermore, modern agentic systems are significantly more complex. Unlike vision classifiers, LLMs work as part of a larger system (i.e., the agent scaffolding, environment, and tools) and this represents a valuable opportunity to incorporate systems security techniques. Similar to how an operating system treats a process as untrusted, we take the stance that the model powering the agent should be treated as untrusted and security properties should be expressed and enforced outside, at the level of the encompassing system.

The computer security community has long studied how to build secure systems from unreliable components [58]. Examples of core principles include the paradigms that software should have the *minimum privilege necessary* to complete its task, it should obey a pre-defined *control and information flow*, and it should *sanitize untrusted data* to avoid being tricked into executing malicious instructions [58]. Today, all major systems implement a subset of these ideas using a variety of mechanisms. Our goal is to distill these principles and mechanisms for the AI security community and to highlight where challenges arise when they are applied to AI agents.

Contributions. We make three main contributions. First, based on our collective experience as cybersecurity researchers working (for multiple decades) on operating systems, network systems, cyber-physical systems, formal methods, programming languages, and adversarial machine learning, we distill core principles that we believe are fundamental for agentic security. Second, we analyze eleven recent and representative real-world attacks on agentic systems and demonstrate how these attacks can be understood, and in many cases mitigated, through the lens of concrete cybersecurity principles. Third, we operationalize these principles and discuss concrete security *mechanisms* that implement them. We believe that if these mechanisms are used when creating agents, a large swath of attacks will be prevented by design. However, as our analysis shows, there are research challenges that arise when applying these security mechanisms to agents. We highlight these challenges, which we suggest the community should focus on.

Overview of system-level mechanisms that mitigate a broad range of agent attacks.

(1) *Provable separation of instructions and data.* Language-based models inherently mix (trusted) instructions and (untrusted) data, creating a fundamental vulnerability whenever malicious data can be interpreted as instructions in the context window. The security community has addressed analogous problems in other systems for decades by enforcing separation of instructions and data at each system layer. Notable examples include the use of no-execute (NX) bits in CPUs and the use of prepared statements and tainting of user-derived bytes to defend against SQL injection in database systems [37, 43]. In agentic systems, the challenges are twofold: first, LLMs combine trusted and untrusted components into a single stream of tokens and process it together, that is, there is no source-level separation between input and data. Second, while it is often desirable for an agent to follow new instructions (once they are separated from data), there should be a deterministic mechanism to decide when it is safe to do so. Even when instructions and data are explicitly delimited [8, 10, 11], persistent attackers have consistently been able to evade the enforcement of separation [26, 36, 42]. Defenses that aim to detect malicious input by identifying instructions embedded in data [30] have likewise failed to be robust to adaptive attacks [13]. We conjecture that model-based defenses for separating instructions and data will always be evaded by persistent attackers and apply the classic security concept of defense-in-depth via the next two primitives.

(2) *Least-privilege sandboxing with verifiable policy generation.* Sandboxing untrusted components is a central principle in systems security, aiming at limiting the scope and impact of a potential attack. Notable examples include isolation and permission mechanisms in operating systems, smartphones and browsers. Each of these restrict access to sensitive resources using least-privilege policies typically specified by a trusted authority (e.g., system administrators or app developers). However, unlike traditional computer programs, agents do not compute fixed tasks. Rather, they can complete different tasks based on the conversation with the user. Therefore, the policy is specified in fuzzy natural language and can change as the agent’s trajectory evolves [67]. This creates fundamental challenges: how to reliably map a natural language policy description into a formal language that is necessary for proper enforcement? how to evolve these policies as the agent’s execution progresses? Towards this end, we argue that security policy generation must be paired with formal verification, such that the resulting policy provably enforces the intended constraints.

(3) *Information flow control.* Once an agent is granted access to sensitive data or resources, it is often necessary to constrain *how* that information is used. Sandboxing only controls whether the agent has access to information in the first place. To bridge this gap, *information flow control* (IFC) mechanisms track data propagation through a system and enforce constraints on its usage. For example, IFC can ensure that sensitive inputs (e.g., location data) do not flow to untrusted outputs (e.g., external network) unless they are anonymized or sanitized. A core tenet is the ability to label data and track its flow through the program, while disentangling multiple flows (often involving other sensitive data in the system). When applied to agents, disentangling flows becomes challenging because LLMs mix all inputs. Thus, the research challenge is disentangling information from different sources as it flows through an AI model.

Scope of the position. Sycophancy, hallucinations, existential risks, and model deception (e.g., scheming, evaluation awareness) are out of scope because they fall within the adjacent, but distinct, AI safety research area, where the model is misaligned and there is no external attacker.

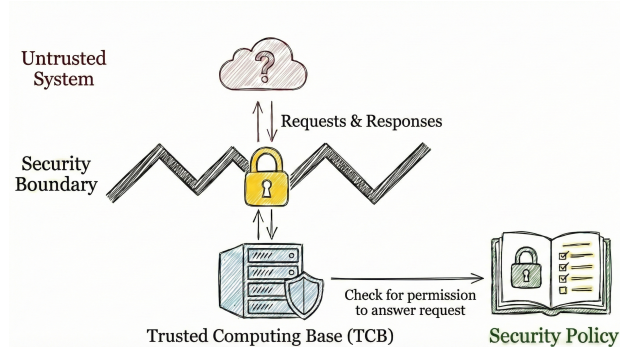


Figure 1: Standard security architecture consists of requests and responses that cross a security boundary between an untrusted system and the trusted computing base (TCB). The TCB consults the security policy to decide whether it is permitted to answer a given request.

2 Challenges in Applying Security Principles to Agentic Systems

We first describe a generic security architecture and list key security principles that have been developed to guide the design and implementation of security architectures. Next, we provide case studies of 2 real attacks on agentic systems which occurred because the agentic systems didn't (or couldn't) operationalize appropriate security principles correctly. An extended list of 9 additional case studies is available in [Appendix A](#). Finally, we surface some challenges in realizing these principles in agentic systems.

2.1 The Security Principles

Abstractly, the standard security architecture ([Figure 1](#)) consists of a *Trusted Computing Base (TCB)*, a *Security Policy*, a *Security Boundary*, and an *Untrusted System*.

The Security Policy is typically a declarative expression of the security goals for a particular system. For example, a security policy might declare which users are allowed to access which files on the computer, or which clients are allowed to query which hosts etc.

While the Security Policy specifies *what* is to be secured, the TCB controls the *how*. The TCB consists of the code and data whose integrity and confidentiality properties cannot be impacted by an attacker, *i.e.*, it is expected to operate correctly even under attack. The TCB typically contains the core functionality of the system (e.g., the kernel of the operating system or the runtime of a programming language) as well as a *Reference Monitor* that approves or rejects requests from various untrusted components to the TCB. The reference monitor is the part of the TCB that parses the security policy and determines whether the next action is approved or rejected.

Next, we list a few key principles that, when correctly implemented in a system, help in providing strong security invariants.

Least Privilege: This principle states that a system component, user, or process should be given only the permissions necessary to perform its intended task, and no more. Systems typically implement this principle by isolating components in a sandbox which can be monitored from outside the sandbox. For example, Android restricts third-party applications from directly accessing parts of the file-system that are owned by other applications.

TCB Tamper Resistance: The TCB of a system should be protected so that attempts by malicious actors to hijack its functioning cannot succeed. For example, operating systems, which act as TCBs for a computer, are protected from malicious modification by hardware features such as a Cryptographic Root of Trust, which verifies and attests that the Operating System being loaded is the correct one.

Complete Mediation: This principle dictates that *every single* request from the Untrusted System must be validated by the reference monitor against the Security Policy. Each time a request crosses the Security Boundary, it must be intercepted by the reference monitor, which in turn must validate the request against the Security Policy. No request is allowed to bypass this check.

Secure Information Flow: While the previous principles cover *access* to information and resources, this principle governs the *long-range flow* of information, requiring that sensitive information does not leak to untrusted areas even via side-channels. This principle requires that the Security Policy must define what kind of information is allowed

to flow in which direction, and the system must dynamically enforce the policy during execution. For instance, even if a request is valid, the TCB must check the Security Policy to ensure that the resulting Response does not contain secret data that the Untrusted System is not authorized to see (this would constitute an unauthorized flow of data from high-trust components to low-trust components).

Human Weak Link: Finally, this principle states that human operators can compromise the above principles in several ways, and thus the security mechanisms in a system must be built with the human in mind. As a user, a human might not understand a permission prompt and allow a malicious program inadvertently. As an administrator, a human could incorrectly configure the Security Policy, making it too permissive (violating Least Privilege). As a developer, a human could accidentally introduce a bug into the Reference Monitor that fails to validate a request correctly (violating Complete Mediation). A good security architecture needs to account for various human failure modes.

2.2 Attacks on Agentic Systems Violate Security Principles

The security principles elaborated above guide the design and implementation of security-critical systems. We present two real-world case studies involving security breaches against agentic systems, and demonstrate how these can be viewed as attacks that violated some security principle. We also present 9 additional case studies in [Appendix A](#). A summary of all studied attacks is in [Table 1](#).

Attack 1: ChatGPT Long-Term Memory SpAIware. A vulnerability in the ChatGPT macOS app allowed persistent data exfiltration by injecting malicious instructions into the app’s “Memories” feature [48]. This was accomplished through a prompt injection in an untrusted document or website, which caused the application to continuously send all of the user’s subsequent conversations to an attacker-controlled server. The data was exfiltrated by rendering an invisible image that included the user’s data as a parameter in the image URL. This attack happened as a consequence of several key security design principle violations.

1. *TCB Tamper Resistance.* In this specific attack, the Memories feature of ChatGPT would presumably represent a trusted source of user’s preferences. As such, it should not have been influenced by third-party data as the third-party data affected the integrity of the core of the system (ChatGPT in this case) directly.
2. *Least Privilege.* In the case of this attack, the system is over-privileged by default and can query arbitrary servers at all times irrespective of whether it was warranted by the user’s requests.
3. *Secure Information Flow.* The non-enforcement of Secure Information Flow allowed the system to transfer information obtained from a trusted domain (ChatGPT Memories) into an untrusted domain (the server).

Attack 2: Claude Code Exfiltration. Claude Code enabled data exfiltration via DNS requests [53]. The attack leveraged indirect prompt injection, where a malicious prompt hidden in a code file instructed Claude Code to read sensitive information, such as API keys from a .env file, and then used an allow-listed command like ping to send that data to an attacker-controlled server as part of a DNS query (via the inherent nslookup that occurs). The result was the leakage of sensitive information from the developer’s machine without consent. Claude Code had implemented human approval for many shell commands that could send data to unknown domains, but mistakenly allowed ping to execute without human approval. The violated principles are:

1. *Least Privilege.* The system violated the principle of least privilege because the agent may not necessarily need uniform access to all shell commands with the ability to supply unrestricted arguments at all points in time.
2. *Secure Information Flow.* Sensitive environment file contents leaked to an unknown domain (in this case the DNS server).

2.3 Agentic Systems Do Not Map Naturally to Established Security Architectures

The two case studies above (and the additional ones in [Appendix A](#)) demonstrate that applying security principles to agentic systems and mapping them to the security architecture of [Figure 1](#) is not straightforward. Developers of these agents attempted to incorporate some principles (e.g., Claude asks for permission) but there were gaps. We identify several research challenges that will benefit from the community’s focus.

The first attack, on ChatGPT, succeeds because the security policy was incomplete and the enforcement mechanism did not guarantee that attacker-controlled data was blocked from being added to trusted storage (in this case, the Memories feature). The three principles involved (TCB Tamper Resistance, Least Privilege, and Secure Information Flow) are well understood but cannot be easily achieved here because **agentic security policies need to be dynamic and task-specific**.

Table 1: The table maps the attacks discussed in [Section 2.2](#) and [Appendix A](#) to the security principles they violate, including least privilege, tamper resistance of the trusted computing base (TCB), complete mediation, secure information flow, and mitigation of human weak links. Checkmarks indicate which principles are implicated by each attack, illustrating that many agentic-AI failures arise from multiple overlapping security breakdowns rather than a single missing defense.

Attack	Least Privilege	TCB Tamper Resistance	Complete Mediation	Secure Information Flow	Human Weak Link
Microsoft Copilot Exfiltration	✓	–	✓	✓	–
Devin AI Exposed Ports	✓	–	–	✓	–
ChatGPT Long-Term Memory SpAIware	–	✓	–	✓	–
Amp AI Code Arbitrary Command Execution	–	✓	–	✓	–
DeepSeek AI Account Takeover	–	–	✓	✓	–
Terminal DiLLMa	–	–	✓	✓	–
ChatGPT Operator Prompt Injection	–	–	–	✓	✓
Devin AI Secret Leaks	✓	–	–	✓	–
Cursor AgentFlayer	✓	–	–	✓	–
Claude Code Exfiltration	✓	–	–	✓	–
AI ClickFix	✓	–	–	✓	✓

In traditional computer security, a security policy governs the privilege of a static piece of code. For example, in Android, the OS developers provide a set of permissions that an app developer can select from. The key aspect is that the app is generally single-purpose and built to perform a fixed set of tasks, and thus, the app developer can create a security policy and present it to the user at the time of installation for permission. In agentic systems, there is no app developer and there is no app. Rather, the user directly provides a natural language task specification to an agent that can evolve over time. Thus, the privilege of the agent in terms of which tools and resources it can access needs to be dynamically adjusted and securely predicted from the description in natural language.

The dynamic requirement of agentic security policies is best illustrated by prompt injection attacks, which leverage the agent’s flexibility to adjust to new instructions in its context window. From the security perspective, dynamic adherence to instructions in non-AI systems is a form of dynamic-code loading. This feature is common among applications that support plugins or extensions, such as OS kernels, IDEs, web browsers, and more. Specifically, dynamic-code loading introduces significant security challenges and has motivated extensive research that can be leveraged in the agentic setting too. The typical solution is to introduce another security boundary between the main TCB and the dynamically loaded code and to use permissions or sandboxing to limit the access afforded to the new code. For example, web pages (probably the most popular platform for dynamic-code loading, as each page load and subsequent interaction can potentially load additional Javascript code as web-page scripts) are secured through a number of sandboxes and associated security policies: Content Security Policy determines which third-party scripts can be loaded, Same-Origin Policy restricts the web page data accessible to a third-party script, `<iframe>` sandboxing isolates third-party content, and Subresource Integrity ensures that third-party scripts have not been modified from the time they were approved by the developer.

In agentic systems, instruction following is a key feature to steer task execution, required by MCP descriptions, Claude skills [2], and OpenClaw’s ClawHub add-ons [39]. Yet current agentic systems lack both of the components that provide security in web page dynamic-code loading. First, information about the source (or provenance) of an instruction given to the agent is hard to assess (in contrast to the source of web-page scripts, most commonly loaded through explicit URLs over HTTPS). Second, sandboxing in the agent context is unavailable or probabilistic at best via mechanisms such as Instruction Hierarchy [10, 68]. An additional challenge to further drive the contrast with the web-page security model is that web pages are designed by a website owner, who is in charge of determining which code to include on the page and indirectly which code to dynamically load at some later time on the page. Agents are often given underspecified instructions and are expected to refine them with additional instructions discovered while working on the task at hand. The capability of agentic systems to accommodate incrementally specified tasks is a highly

desirable property, mirroring the execution of incrementally constructed programs in REPLs. This flexibility, however, means that security mechanisms are not readily available, typically falling through to OS-level permission boundaries; agentic systems inherit the same gap.

A potential solution to the fluid nature of agentic security policies is to use a safety LLM to predict the security policy and to reason in real time through a security decision, but such a solution is unsatisfactory as **safety agents are unpredictably probabilistic reference monitors and TCBs**. In the classic systems scenario, the predictability of the TCB is an explicit assumption, and is guaranteed either through the deterministic behavior of reference monitors or through probabilistic bounds backed by hardness assumptions of cryptographic protocols. In contrast, the TCB in agentic systems becomes probabilistic when LLMs are used to predict security policies, as there is no guarantee that an LLM-generated policy will be able to satisfy the principles of Least Privilege and Contextual Integrity. Unlike traditional TCB components built on deterministic code, where security properties can, in principle, be formally verified against a precise specification, an LLM-based policy predictor does not provide a precise contract for enforcement. Differently put, there is no formal specification of which actions should be allowed or denied under which histories, and therefore no way to prove that the LLM will consistently implement the intended policy. Similarly, using an additional LLM as a reference monitor remains vulnerable to adversarial inputs designed to exploit gaps in security reasoning.

Even if we had a perfectly aligned LLM that generated a correct and least privileged security policy, identifying the correct layer at which security policies are enforced is challenging due to the **semantic gap at the security boundary**. The policy is expressed at a different abstraction level than where it can be enforced. For instance, agents access tools that cover a whole range of system layers, from simple command-line tools, to powerful shells and interpreters, as well as APIs and UIs for a variety of services. Enforcing a policy that covers all of these is non-trivial, yet crucial to achieve complete mediation (ensuring that all ways to perform an operation face a security check). In traditional computer systems, we have layered architectures (Hardware $\Leftrightarrow$ Operating System Kernel $\Leftrightarrow$ Process $\Leftrightarrow$ Network) that afford various granularities of abstraction for enforcing security policies. These layers allow practitioners to use contextually appropriate semantic information to inform enforcement decisions. For example, the OS Kernel can restrict which processes, such as a browser, can write to which files and directories, whereas the browser controls what cookies are sent to websites. The latter represents a finer and more context-dependent decision than the coarse control enforced by an OS. Put another way, the OS does not have visibility into the semantics of a higher layer because it exists at the lower layer in the abstraction stack. Agentic systems lack these layers. Rather, they take very high-level prompt instructions and convert them to low-level actions such as tool calls. This lack of abstraction layers results in the semantic gap.

3 Open Research Problems

We identify three concrete challenges that, if addressed appropriately and systematically, can solve a significant number of security and privacy issues in agentic systems.

3.1 Separating Instructions from Data

Instruction-data separation is a cornerstone of modern operating systems security, ensuring TCB tamper resistance and control flow integrity. Specifically, hardware functionality allows the operating system to mark regions of memory as writable or executable, but not both (often referred to as $W \oplus X$). Placing code in executable-but-not-writable memory and data in writable-but-not-executable memory prevents buffer-overflow attacks that attempt to treat data received from an untrusted source as instructions to be executed.¹ The analogous case in agentic systems is the need to separate instructions from data in the LLM’s context window, which typically mixes the two. By doing so, many prompt-injection attacks can be prevented, as in this setting, the attacker plants malicious instructions within untrusted data sources (e.g., emails, calendar events, webpages, desktop notifications) [24]. This, in turn, raises several research questions: (1) What is a precise and formal definition of separating instructions and data [72]? (2) Given a definition, is it practical to construct (in terms of capabilities, scale, cost) an LLM that provably follows this separation? (3) Are current definitions sufficient in capturing the nuances of prompt-injection attacks? (4) How does one obtain the required dynamic behavior?

One popular interpretation is for the agent/LLM to not follow any instructions that appear in the context window tagged as “data” [8–10, 24, 68, 70]. However, such definitions typically cannot be enforced provably, and most current approaches are best-effort heuristics. Furthermore, prompt injection attacks have been shown in both the black-box

¹We note that $W \oplus X$ does not completely remove code execution vulnerabilities due to advanced techniques like return-to-libc or return-oriented programming [60], but is nonetheless foundational in systems security.

and white-box setting [36, 42, 44], as one can trick the model into following instructions despite any fine-tuning the model might have undergone to learn tag separators. Furthermore, with multi-modal becoming standard, instructions may appear not only in text, but also in images, video, and audio. The past decade of adversarial machine learning has shown that “continuous domain” adversarial examples are easy to carry out [5]. As a result, a probabilistic separation of instructions and data allows the attacker to employ an optimizer to easily find counter-examples to break such defenses.

Furthermore, this policy might be too restrictive for many tasks, as one of the key benefits of agents is their ability to act upon new information to complete an (often underspecified) goal. For example, learning to use an API and following web links based on information present in a web page are crucial to such adaptability, though they involve some form of following new instructions present in data. A more nuanced policy requires enforcement that must first identify new instructions inside the data and then decide whether such instructions should be followed. In traditional systems security, policies such as Control Flow Integrity (CFI) provide a foundation that specifies whether to execute new instructions once they are identified, but no equivalent policy exists for agentic systems.

We pose the challenge of separating instructions and data (if attainable) as a key research problem to pursue as it will mitigate a large class of prompt injection attacks. However, we note that separating instructions and data is unlikely to fully solve the prompt injection problem, as adversarial data may poison agentic execution (for example, via the arguments to an agent’s tools) even if the agent does not follow external instructions.

3.2 Verifiable Policy Generation

A security policy specifies which actions a system component is authorized to perform, and a reference monitor consults it on every request to render an approve-or-deny decision. This design underpins least-privilege enforcement in operating systems, browsers, mobile platforms, and cloud authorization frameworks [1, 15, 19, 23, 33]. Provable enforcement guarantees rest on two structural conditions: the policy is expressed in a formal language with precise semantics, and it typically ranges over a fixed action space, such as system calls, API endpoints, or platform intents. Agentic deployments satisfy neither condition. Both failures stem from the absence of layered abstraction boundaries, an instance of the semantic gap problem discussed in Section 2.3. Agents collapse high-level user intent into low-level tool calls without intervening layers to host either a fixed action vocabulary or layer-native policy predicates. The action space is open: agents compose tools at runtime, traverse arbitrary URLs, and emit unbounded sequences of operations, so even an expert author cannot anticipate every action that a formal policy must constrain. The policy content is also intrinsically semantic: constraints such as “do not share sensitive data” or “do not take harmful actions” invoke predicates (*sensitive* and *harmful*) whose interpretation is context-dependent and lies beyond the expressivity of formal policy languages. Agentic policies, therefore, reside in natural language: user prompts, system prompts, and regulations. They are also dynamic, evolving as the agent’s task unfolds and new sub-goals emerge. Unlike in classical systems, the model can consume such artifacts directly. From a security standpoint, however, this is the wrong choice. The principled approach is to *construct a translation pipeline that automatically converts natural-language artifacts into formal, enforceable least-privilege policies with verifiable correctness*, thereby relocating enforcement from the untrusted model to a deterministic reference monitor.

Prior work proceeds along two lines. The first concerns deterministic enforcement given a formal policy: recent systems [7, 14, 17, 22, 29, 32, 38, 40, 61, 65] provide policy languages spanning capabilities, information-flow labels, and programmable rails, each yielding strong runtime guarantees *conditioned on the correctness of the supplied policy*. The second concerns translating natural-language sources into formal policies [12, 34, 61]; existing approaches typically delegate this translation to an unvalidated LLM call, leaving the policy without a correctness guarantee. Bridging the two requires a translator with a precise correctness criterion and a verification procedure.

The natural correctness criterion is mutual entailment: the natural-language source N and the formal policy P must agree on authorization decisions across all accessible states of the deployment. This criterion is not directly checkable, since natural language admits ambiguity at every level. Consequently, any automated procedure that adjudicates entailment is itself probabilistic, motivating relaxation of the criterion along three axes, each constituting an open research problem. The first concerns *grounding*: mutual entailment between N and P is well-posed only relative to the *substrate* [40] of named predicates and interpretations against which P is evaluated. This substrate must encode the agent’s action space and the concepts to which N refers but rarely specifies, including what qualifies as *sensitive*, or which sites are *trusted*. The second concerns *checkability*: the entailment check is probabilistic even when the substrate is fixed. The question is whether structural constraints on N and P (for example, restricted policy fragments or schemas for the natural-language source) can make this check reliable enough that combining it with deterministic enforcement yields a meaningful end-to-end guarantee. The third concerns the *synthesis procedure*: translators from N to P must

come with a measurable bound on the residual divergence between P and N , even when strong mutual entailment is unattainable, and must support incremental refinement as N evolves over the agent’s task.

Verifiable policy generation bridges natural-language intent and formal enforcement, extending deterministic guarantees to agentic deployments. While progress on any of the axes (grounding, checkability, or synthesis procedure) is meaningful, closing the gap fully requires advances along all three.

3.3 Information-Flow Control (IFC)

Access control as described above is a “gate” because it only gives a yes/no answer on whether the agent should have access to resources (one or more tools). In contrast, Information Flow Control (IFC) is a well-studied security primitive that helps reason about long-range flow of information across a system by tagging the provenance of objects flowing through a system.

IFC works by labeling data, then tracking those labels (statically or dynamically as the program executes), and enforcing label-based policies as the program operates on the data [18, 20, 66]. This labeling and tracking can be done at multiple levels of granularity (e.g., processor-instruction level, program-variable level, process level, filesystem-level, cross-computer level), affording various safety guarantees. However, the common assumption is that it is possible to track labels as the corresponding data makes its way through the system and update the labels accordingly, through *label propagation rules* [18] that describe how data can flow through the system.

Performing label propagation on LLMs is an open problem. Whenever multiple data values from multiple sources (and their corresponding labels) are concatenated and fed into the model, currently we can only assume that the LLM’s output is labeled with the union of those labels [14].

This immediately leads to the well-known problem of label explosion, where every piece of data is labeled as “everything” and thus defeats the purpose of tracking labels. Three potential directions can address this challenge. First, by replacing hard labels with soft distributional labels in the style of *quantitative information flow* [35, 63]: each output carries a set of fuzzy labels quantifying how much information it carries about the labeled input data. Second, by taking a *causal-interventional* view [6, 16, 25]: quantifying the causal effect of a set of inputs by intervening on them, such as by removal or perturbation, and measuring the effect on the distribution of actions. Third, extending these interventional methods into the model’s internals to obtain a faithful *mechanistic interpretation* [21, 41], which provides an equivalent algorithmic proxy of the LLM on which IFC can be applied.

A second related challenge is downgrading of labeled data: *declassification*, marking labeled sensitive data as safe to release to untrusted recipients, and *endorsement*, treating untrusted data as trusted. Correct downgrading can likewise resolve the label explosion, but has been a persistent open problem in IFC for nearly two decades [57]. Human-in-the-loop techniques are commonly used for policy overrides [56]; recent work [28] extends this to endorsement with taint-aware planning which minimizes sensitive action attempts in a tainted state, and hence minimizes the need for approvals. Combinations of established downgrading techniques with minimal human-in-the-loop overrides may enable practical downgrading in IFC-enforced agentic systems.

A third challenge is the granularity of labeling: standard IFC theory specifies fine-grained (variable-level or memory-level) and coarse-grained labeling (process-level, file-level, module-level) corresponding to various levels of abstraction in a traditional computer system. By contrast, the input to an LLM is a sequence of tokens with no concrete natural structure: in another instance of the semantic gap problem discussed in Section 2.3, agents lack the layered abstraction boundaries that give traditional systems their natural labeling granularities. Without natural abstraction boundaries to anchor labels, practical choices must be derived from the available structure. One option is to re-use the existing context labeling of system, user and tool that are commonplace in many chat formats. Another option might be to leverage domain-specific knowledge. For example, a web agent imposes a structure on the context window, e.g., some sequences of tokens naturally belong to a web origin, and thus, one can perform labeling at that granularity.

In summary, we believe that IFC labeling layers should be added beyond least-privilege access control. However, this requires solving the fundamental challenge of label assignment granularity and precise label propagation in LLMs.

4 Objections and Counter-arguments

The primary objection to our thesis stems from a model-centric perspective, which posits that achieving near-total model robustness against adversarial attacks is the only necessary safeguard. Proponents of this view often argue that traditional systems-security methods are too invasive, requiring significant architectural overhauls and inducing

unacceptable overheads for both performance and utility. From their vantage point, focusing on the model alone offers a more streamlined deployment path.

We acknowledge that security inherently involves a trade-off between engineering effort, system performance, and utility. However, from a rigorous security standpoint, the model-centric approach faces two fundamental hurdles. **(1) Intent Ambiguity:** Defining “100% robustness” is conceptually fraught. It requires a model to perfectly adjudicate a user’s latent intent against an instruction’s potential for harm. Defining this boundary is often context-dependent and linguistically fluid. **(2) Non-deterministic Fragility:** LLMs remain fundamentally non-deterministic, even when operating at zero temperature. This inherent variance means that slight perturbations in input can lead to drastically different behavioral outcomes. We believe that such unpredictability is unacceptable when managing critical system properties, such as data deletion or the exfiltration of sensitive information.

A secondary counter-argument suggests that a systems-level architecture can be simulated by composing multiple ML-based components—such as input/output classifiers or constitutional filters. While this creates a multi-layered defense, we argue that merely stacking ML models does not constitute true defense-in-depth. These “guard” models often share the same statistical failure modes as the primary agents they monitor [31]. Furthermore, without a precise TCB that is formally specified, correctness cannot be rigorously proved.

Although model composition does raise the bar for attackers, they can frequently orchestrate attacks that bypass composed defenses [31] by exploiting universal prompt injections [3, 71], training-data poisoning [4], or adversarial fine-tuning [59]. Because these layers rely on correlated training distributions and learned representations, their failures are not independent; an exploit that fools the agent is statistically likely to fool its ML-based monitor.

In contrast, our proposed systems-oriented approach introduces *heterogeneous enforcement*. By deploying mechanisms that operate at different layers of abstraction, such as least-privilege sandboxing, strict instruction/data separation, and deterministic information flow tracking, we force an attacker to bypass multiple, mechanistically distinct barriers. As a result, an attacker may need to circumvent multiple distinct defenses rather than exploit a single shared statistical weakness.

5 Conclusion

Model-centric approaches to agent security are unlikely to be dependable when faced with powerful adversaries. We propose a systems-driven approach, which hardens a system as a whole, despite unreliable and imperfect individual components. Towards this end, we propose viewing agent security as an instance of computer security. This domain has long dealt with powerful attackers and motivated decades of research on principles and techniques that deal with such adversaries. We also demonstrate how recent attacks can be viewed through the lens of computer security, and present classic design principles from system security research that are relevant to the agentic setting. Specifically, we cover three main security mechanisms: (1) provable instruction and data separation; (2) verifiable least-privilege policy generation; and (3) information flow control. Based on the attack case studies, we postulate that applying these three mechanisms will eliminate a large fraction of attacks, but doing so will require resolving the challenges we outlined. We call on the community to view agent security as a systems problem and to tackle the highlighted research challenges.

References

- [1] Amazon Web Services. AWS Identity and Access Management (IAM). <https://aws.amazon.com/iam/>. Accessed: 2026.
- [2] Anthropic. Agent skills—Claude API docs, 2026. Accessed: 2026-02-05. URL: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>.
- [3] Anthropic. Next-generation constitutional classifiers: More efficient protection against universal jailbreaks. <https://www.anthropic.com/research/next-generation-constitutional-classifiers>, January 2026.
- [4] Anthropic. Poisoning fine-tuning datasets of constitutional classifiers. <https://alignment.anthropic.com/2026/backdooring-classifiers/>, April 2026.
- [5] Anish Athalye, Nicholas Carlini, and David Wagner. Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples, 2018. URL: <https://arxiv.org/abs/1802.00420>, arXiv: 1802.00420.

- [6] Nihat Ay and Daniel Polani. Information flows in causal networks. *Advances in complex systems*, 11(01):17–41, 2008.
- [7] Luca Beurer-Kellner, Marc Fischer, Hemang Sarkar, Kristian Bonde Nielsen, Marco Milanta, and Aleksei Kudrinskii. Introducing Guardrails: The Contextual Security Layer for the Agentic Era. <https://invariantlabs.ai/blog/guardrails>, April 2025. Accessed: 2026-02-05.
- [8] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2383–2400, 2025. URL: <https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe>.
- [9] Sizhe Chen, Yizhu Wang, Nicholas Carlini, Chawin Sitawarin, and David Wagner. Defending against prompt injection with a few defensive tokens, 2025. URL: <https://arxiv.org/abs/2507.07974>, arXiv:2507.07974.
- [10] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending Against Prompt Injection with Preference Optimization. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2025.
- [11] Sizhe Chen, Arman Zharmagambetov, David Wagner, and Chuan Guo. Meta secalign: A secure foundation llm against prompt injection attacks, 2026. URL: <https://arxiv.org/abs/2507.02735>, arXiv:2507.02735.
- [12] Zhaorun Chen, Mintong Kang, and Bo Li. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning, 2025. URL: <https://arxiv.org/abs/2503.22738>, arXiv:2503.22738.
- [13] Sarthak Choudhary, Divyam Anshumaan, Nils Palumbo, and Somesh Jha. How Not to Detect Prompt Injections with an LLM. In *Proceedings of the 18th ACM Workshop on Artificial Intelligence and Security*, pages 218–229, 2025.
- [14] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI Agents with Information-Flow Control, 2025. URL: <https://arxiv.org/abs/2505.23643>, arXiv:2505.23643.
- [15] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Runpta, Emina Torlak, and Andrew M. Wells. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proc. ACM Program. Lang.*, 8(OOPSLA1), April 2024. doi:10.1145/3649835.
- [16] Anupam Datta, Shayak Sen, and Yair Zick. Algorithmic transparency via quantitative input influence: Theory and experiments with learning systems. In *2016 IEEE symposium on security and privacy (SP)*, pages 598–617. IEEE, 2016.
- [17] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating Prompt Injections by Design, 2025. URL: <https://arxiv.org/abs/2503.18813>, arXiv:2503.18813.
- [18] Dorothy E. Denning. A Lattice Model of Secure Information Flow. *Commun. ACM*, 19(5):236–243, May 1976. doi:10.1145/360051.360056.
- [19] John DeTreville. Binder, a Logic-based Security Language. In *Proceedings 2002 IEEE Symposium on Security and Privacy*, pages 105–113. IEEE, 2002.
- [20] William Enck, Peter Gilbert, Seungyeop Han, Vasant Tendulkar, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. *ACM Trans. Comput. Syst.*, 32(2), June 2014. doi:10.1145/2619091.
- [21] Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. *Advances in neural information processing systems*, 34:9574–9586, 2021.
- [22] Google. CAPSEM: Contextual Agent Privacy and Security Manager. <https://capsem.org/>, 2026. Accessed: 2026-02-05.

- [23] Google Cloud. Identity and Access Management (IAM). <https://cloud.google.com/iam/>. Accessed: 2026.
- [24] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what You’ve Signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection, 2023. URL: <https://arxiv.org/abs/2302.12173>, arXiv:2302.12173.
- [25] Dominik Janzing, David Balduzzi, Moritz Grosse-Wentrup, and Bernhard Schölkopf. Quantifying causal influences. 2013.
- [26] Yuqi Jia, Zedian Shao, Yupei Liu, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. A critical evaluation of defenses against prompt injection attacks. In *34th USENIX Security Symposium (USENIX Security 25)*, 2025.
- [27] Sam Johnson, Viet Pham, and Thai Le. Manipulating llm web agents with indirect prompt injection attack via html accessibility tree. *arXiv preprint arXiv:2507.14799*, 2025.
- [28] Aashish Kolluri, Rishi Sharma, Manuel Costa, Boris Köpf, Tobias Nießen, Mark Russinovich, Shruti Tople, and Santiago Zanella-Beguelin. Optimizing agent planning for security and autonomy. In *The Fourteenth International Conference on Learning Representations*, 2026. URL: <https://openreview.net/forum?id=g0aVCDY3gS>.
- [29] Evan Li, Tushin Mallick, Evan Rose, William Robertson, Alina Oprea, and Cristina Nita-Rotaru. ACE: A Security Architecture for LLM-Integrated App Systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2026.
- [30] Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 2190–2208, 2025. doi:10.1109/SP61157.2025.00250.
- [31] Neal Mangaokar, Ashish Hooda, Jihye Choi, Shreyas Chandrashekar, Kassem Fawaz, Somesh Jha, and Atul Prakash. Prp: Propagating universal perturbations to attack large language model guard-rails, 2024. URL: <https://arxiv.org/abs/2402.15911>, arXiv:2402.15911.
- [32] Luoxi Meng, Henry Feng, Ilia Shumailov, and Earlene Fernandes. ceLLMate: Sandboxing Browser AI Agents, 2025. URL: <https://arxiv.org/abs/2512.12594>, arXiv:2512.12594.
- [33] Microsoft. Azure Policy Documentation. <https://learn.microsoft.com/en-us/azure/governance/policy/>. Accessed: 2026.
- [34] Lesly Miculicich, Mihir Parmar, Hamid Palangi, Krishnamurthy Dj Dvijotham, Mirko Montanari, Tomas Pfister, and Long T. Le. VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation, 2025. URL: <https://arxiv.org/abs/2510.05156>, arXiv:2510.05156.
- [35] S Alvim M’rio, Kostas Chatzikokolakis, Catuscia Palamidessi, and Geoffrey Smith. Measuring information leakage using generalized gain functions. In *2012 IEEE 25th Computer Security Foundations Symposium*, pages 265–279. IEEE, 2012.
- [36] Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilia Shumailov, Abhradeep Thakurta, Kai Yuanqing Xiao, Andreas Terzis, and Florian Tramèr. The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against Llm Jailbreaks and Prompt Injections, 2025. URL: <https://arxiv.org/abs/2510.09023>, arXiv:2510.09023.
- [37] Anh Nguyen-Tuong, Salvatore Guarnieri, Doug Greene, Jeff Shirley, and David Evans. Automatically hardening web applications using precise tainting. In *IFIP International Information Security Conference*, pages 295–307. Springer, 2005.
- [38] NVIDIA. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. <https://github.com/NVIDIA/NeMo-Guardrails>, 2023. Accessed: 2026-02-03. URL: <https://github.com/NVIDIA/NeMo-Guardrails>.
- [39] OpenClaw. ClawHub, the skill dock for sharp agents, 2026. Accessed: 2026-02-05. URL: <https://clawhub.ai/>.

- [40] Nils Palumbo, Sarthak Choudhary, Jihye Choi, Guy Amir, Prasad Chalasani, and Somesh Jha. Formal policy enforcement for real-world agentic systems, 2026. URL: <https://arxiv.org/abs/2602.16708>, arXiv:2602.16708.
- [41] Nils Palumbo, Ravi Mangal, Zifan Wang, Saranya Vijayakumar, Corina S Pasareanu, and Somesh Jha. Validating mechanistic interpretations: An axiomatic approach. In *International Conference on Machine Learning*, pages 47509–47544. PMLR, 2025.
- [42] Nishit V. Pandya, Andrey Labunets, Sicun Gao, and Earlence Fernandes. May I have your Attention? Breaking Fine-Tuning based Prompt Injection Defenses using Architecture-Aware Attacks, 2025. URL: <https://arxiv.org/abs/2507.07417>, arXiv:2507.07417.
- [43] Tadeusz Pietraszek and Chris Vanden Berghe. Defending against injection attacks through context-sensitive string evaluation. In *International Workshop on Recent Advances in Intrusion Detection*, pages 124–145. Springer, 2005.
- [44] J. Rehberger (wunderwuzzi). Breaking Instruction Hierarchy in OpenAI’s gpt-4o-mini. <https://embracethered.com/blog/posts/2024/chatgpt-gpt-4o-mini-instruction-hierarchy-bypasses/>, July 2024. Accessed: 2025-11-03.
- [45] J. Rehberger (wunderwuzzi). DeepSeek AI: From prompt injection to account takeover. <https://embracethered.com/blog/posts/2024/deepseek-ai-prompt-injection-to-xss-and-account-takeover/>, 2024. Accessed on 2025-09-05.
- [46] J. Rehberger (wunderwuzzi). Google Gemini: Planting Instructions For Delayed Automatic Tool Invocation, feb 2024. URL: <https://embracethered.com/blog/posts/2024/llm-context-pollution-and-delayed-automated-tool-invocation/>.
- [47] J. Rehberger (wunderwuzzi). Microsoft Copilot: From Prompt Injection to Exfiltration of Personal Information. <https://embracethered.com/blog/posts/2024/m365-copilot-prompt-injection-tool-invocation-and-data-exfil-using-ascii-smuggling/>, 2024. Accessed on 2025-09-05.
- [48] J. Rehberger (wunderwuzzi). Spyware Injection Into Your ChatGPT’s Long-Term Memory (SpAIware). <https://embracethered.com/blog/posts/2024/chatgpt-macos-app-persistent-data-exfiltration/>, 2024. Accessed on 2025-09-05.
- [49] J. Rehberger (wunderwuzzi). Terminal DiLLMas—Prompt Injection in the Terminal via ANSI Sequences. <https://embracethered.com/blog/posts/2024/terminal-dillmas-prompt-injection-ansi-sequences/>, 2024. Accessed on 2025-09-05.
- [50] J. Rehberger (wunderwuzzi). AI ClickFix: Hijacking Computer-Use Agents Using ClickFix. Blog post, May 2025. URL: <https://embracethered.com/blog/posts/2025/ai-clickfix-ttp-claude/>.
- [51] J. Rehberger (wunderwuzzi). AMP—Agents that Modify System Configuration and Escape. <https://embracethered.com/blog/posts/2025/amp-agents-that-modify-system-configuration-and-escape/>, 2025. Accessed on 2025-09-05.
- [52] J. Rehberger (wunderwuzzi). ChatGPT Operator prompt injection exploits. <https://embracethered.com/blog/posts/2025/chatgpt-operator-prompt-injection-exploits/>, 2025. Accessed on 2025-09-05.
- [53] J. Rehberger (wunderwuzzi). Claude Code: Data Exfiltration with DNS (CVE-2025-55284). Blog post, August 2025. URL: <https://embracethered.com/blog/posts/2025/claude-code-exfiltration-via-dns-requests/>.
- [54] J. Rehberger (wunderwuzzi). Devin AI Kill Chain—Exposing Ports Leading to RCE and file Exfiltration. <https://embracethered.com/blog/posts/2025/devin-ai-kill-chain-exposing-ports/>, 2025. Accessed on 2025-09-05.
- [55] J. Rehberger (wunderwuzzi). Devin can leak your secrets—Prompt Injection Leads to Exfiltration. <https://embracethered.com/blog/posts/2025/devin-can-leak-your-secrets/>, 2025. Accessed on 2025-09-05.

- [56] Franziska Roesner, Tadayoshi Kohno, Alexander Moshchuk, Bryan Parno, Helen J Wang, and Crispin Cowan. User-driven access control: Rethinking permission granting in modern operating systems. In *2012 IEEE Symposium on Security and Privacy*, pages 224–238. IEEE, 2012.
- [57] Andrei Sabelfeld and David Sands. Declassification: Dimensions and principles. *Journal of Computer Security*, 17(5):517–548, 2009.
- [58] J.H. Saltzer and M.D. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi:10.1109/PROC.1975.9939.
- [59] Bilgehan Sel, Xuanli He, Alwin Peng, Ming Jin, and Jerry Wei. Trojan-speak: Bypassing constitutional classifiers with no jailbreak tax via adversarial finetuning, 2026. URL: <https://arxiv.org/abs/2603.29038>, arXiv:2603.29038.
- [60] Hovav Shacham. The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86). In *Proceedings of the 14th ACM Conference on Computer and Communications Security, CCS ’07*, page 552–561, New York, NY, USA, 2007. Association for Computing Machinery. doi:10.1145/1315245.1315313.
- [61] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Programmable Privilege Control for LLM Agents, 2025. URL: <https://arxiv.org/abs/2504.11703>, arXiv:2504.11703.
- [62] Marina Simakov. AgentFlayer: When a Jira Ticket Can Steal Your Secrets. <https://labs.zenity.io/p/when-a-jira-ticket-can-steal-your-secrets>, August 2025. Accessed: 2025-09-17.
- [63] Geoffrey Smith. On the foundations of quantitative information flow. In *International Conference on Foundations of Software Science and Computational Structures*, pages 288–302. Springer, 2009.
- [64] Georgios Syros, Evan Rose, Brian Grinstead, Christoph Kerschbaumer, William Robertson, Cristina Nita-Rotaru, and Alina Oprea. Muzzle: Adaptive agentic red-teaming of web agents against indirect prompt injection attacks. *arXiv preprint arXiv:2602.09222*, 2026.
- [65] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. SAGA: A Security Architecture for Governing AI Agentic Systems. In *Network and Distributed System Security Symposium (NDSS)*, 2026.
- [66] Trishita Tiwari, Suchin Gururangan, Chuan Guo, Weizhe Hua, Sanjay Kariyappa, Udit Gupta, Wenjie Xiong, Kiwan Maeng, Hsien-Hsin S. Lee, and G. Edward Suh. Information flow control in machine learning through modular model architecture. In *Proceedings of the 33rd USENIX Conference on Security Symposium, SEC’24, USA, 2024*. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/tiwari>.
- [67] Udbhav Tiwari and Meredith Whittaker. AI Agent, AI Spy. <https://media.ccc.de/v/39c3-ai-agent-ai-spy>. 39th Chaos Communication Congress, Congress Center Hamburg, Hamburg, Germany. URL: <https://media.ccc.de/v/39c3-ai-agent-ai-spy>.
- [68] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions, 2024. URL: <https://arxiv.org/abs/2404.13208>, arXiv:2404.13208.
- [69] Zhun Wang, Vincent Siu, Zhe Ye, Tianneng Shi, Yuzhou Nie, Xuandong Zhao, Chenguang Wang, Wenbo Guo, and Dawn Song. Agentvigil: Generic black-box red-teaming for indirect prompt injection against llm agents. *arXiv preprint arXiv:2505.05849*, 2025.
- [70] Tong Wu, Shujian Zhang, Kaiqiang Song, Silei Xu, Sanqiang Zhao, Ravi Agrawal, Sathish Reddy Indurthi, Chong Xiang, Prateek Mittal, and Wenxuan Zhou. Instructional Segment Embedding: Improving LLM Safety with Instruction Hierarchy, 2025. URL: <https://arxiv.org/abs/2410.09102>, arXiv:2410.09102.
- [71] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models, 2023. URL: <https://arxiv.org/abs/2307.15043>, arXiv:2307.15043.

- [72] Egor Zverev, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, and Christoph H. Lampert. Can LLMs Separate Instructions From Data? And What Do We Even Mean By That? In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL: <https://iclr.cc/virtual/2024/23872>.

A Attacks on Agentic Systems: Additional Case Studies

DeepSeek AI Account Takeover. A vulnerability in the DeepSeek AI platform led to a full account takeover by chaining a prompt injection with a Cross-Site Scripting (XSS) exploit [45]. An attacker uploaded a malicious text file containing a base64-encoded JavaScript payload, which, when processed by a victim’s account, instructed the AI to decode and execute the script in the victim’s browser, stealing the `userToken` from `localStorage` and sending it to the attacker.

Violation: This attack violates the principles of Secure Information Flow (as it established an unauthorized covert channel to leak a sensitive session token from the user’s browser to an external, malicious endpoint, completely bypassing the platform’s intended data handling and security boundaries) and *Complete Mediation* (as the agent processed encoded data meant to bypass input filtering). Chained code injections are often used in traditional exploits to complete an attack.

Defense: Ensure a strict separation between data (uploaded text files) and executable web code. The fuzzy security boundary coupled with the need to allow for some flexibility in using external data as prompts (a form of dynamic code loading) makes it challenging to secure the agentic system end-to-end when no good mechanisms for separating instruction and data are available (see [Section 3.1](#)).

Terminal DiLLMa. The “Terminal DiLLMa” attack hijacked a user’s terminal through LLM-powered command-line tools by using prompt injection to generate malicious ANSI escape sequences [49]. When a compromised tool, such as the proof-of-concept `dillma.py`, processed a malicious prompt, it output specially crafted ANSI codes that the terminal emulator executes, leading to unauthorized actions like clipboard manipulation or data exfiltration via DNS requests.

Violation: This attack highlights a violation of the Complete Mediation and Secure Information Flow principles. Insufficient or incorrect input validation and sanitization result in incompletely validated inputs which then trigger unwanted software behaviors, as occurred here.

Defense: Implement a sanitization layer for agent output for dangerous ANSI escape sequences before they reach the terminal emulator. This becomes challenging when different types of outputs can be created (e.g., with ANSI escape codes, with Markdown formatting, with HTML tags, and with JavaScript) and can be displayed in various settings (e.g., terminals, browsers, text viewers).

AI ClickFix. Traditional social-engineering techniques were adapted to use against computer-use agents, in an attack called “AI ClickFix” [50]. The attack involved tricking an AI agent into executing malicious code by presenting it with a series of instructions on a webpage. For instance, the agent was prompted to click a button which secretly copied a malicious command to the clipboard; then, the agent was instructed to open a terminal and paste the command from the clipboard into the terminal. This resulted in the AI system being hijacked to execute arbitrary commands from untrusted web content.

Violation: This attack demonstrates that AI agents can be “socially engineered,” violating the principles of Human Weak Link, Least Privilege, and Secure Information Flow.

Defense: Introduce a hard security boundary between untrusted web content and privileged system tools. The agent should not be allowed to copy data from a browser clipboard directly into a system terminal without user consent. This would, however, restrict the agent’s functionality and burden the user with security decisions, effectively requiring the user to determine what is an appropriate (visual) instruction and what is data ([Section 3.1](#)).

Microsoft Copilot Exfiltration. A vulnerability in Microsoft 365 Copilot allowed attackers to steal private user data, such as emails, by sending a malicious message containing a hidden prompt [47]. When a user interacted with this message (asking, for example, for a summary) via Copilot, a prompt-injection attack was triggered, letting the attacker take control of the agent. The compromised Copilot was then instructed to find sensitive information, encode it using “ASCII smuggling,” and embed it into a seemingly harmless hyperlink. When the user clicked this link, their data was secretly sent to the attacker.

Violation: This attack violates the security principles of *Least Privilege*, *Complete Mediation*, and *Secure Information Flow*, as Copilot automatically performed unexpected actions sourced from a document of unknown origin (like searching for and exfiltrating data) without verifying each step with the user, failing to check that the AI’s operations were fully authorized. This vulnerability is similar to that of traditional code injection after a buffer overflow, where an adversarially crafted input allows the attacker to run code of their choice inside the victim process, coupled with insufficient access control.

Defense: Implement strict output sanitization to detect and block data ex-filtration channels like “ASCII smuggling” in generated hyperlinks. Require human approval whenever the agent accesses sensitive data. Because the TCB is probabilistic (the Copilot LLM is part of the TCB), and the security boundary for Internet access is fuzzy (the set of safe URLs cannot be practically enumerated in full), these defenses provide only incomplete security guarantees and will need to be supplemented with mechanisms for separating instructions and data and for least-privilege access control.

Cursor AgentFlayer. A malicious Jira ticket was used to trick the AI-powered code editor, Cursor, into exfiltrating sensitive information [62]. The attack created a Jira ticket with a prompt that, while seemingly harmless, caused Cursor to leak repository secrets or even personal files, like Amazon Web Services (AWS) credentials, from the user’s local system. The author shows how simple changes in wording bypassed the AI’s built-in security measures. This exploit required, as a prerequisite, that a developer disabled the human-in-the-loop validation for the Jira MCP server or entirely enabled Cursor’s Auto Run mode (a form of YOLO mode for agents where confirmation prompts are minimized, often used to give the agents more freedom without involving the developer).

Violation: This exploit violates the Secure Information Flow, Least Privilege, Complete Mediation, and TCB Tamper Resistance principles.

Defense: Implement fine-grained file system permissions, restricting the agent to only the specific repositories or directories it is currently working on. However, the set of required permissions might depend on the prompt as overly restrictive permissions could impact utility. Similar to the preceding attack, a defense that guarantees least-privilege access is desirable but hard to realize practically.

Devin AI Exposed Ports. The AI agent Devin comes with a tool called `expose_port`, meant for testing, that was abused through an indirect prompt injection attack [54]. An attacker hosted a malicious prompt on a website that, when visited by Devin, hijacked the agent. The compromised AI then started a local web server, exposing its entire file system, and used the `expose_port` tool to make this server publicly accessible online. The resulting URL was then sent to the attacker, granting them full access to Devin’s files.

Violation: This vulnerability violates the principles of Least Privilege (as the `expose_port` tool had excessive permissions, allowing it to expose any port, including one with access to the entire file system, rather than being restricted to only what was necessary for its intended function) and of Secure Information Flow (as the agent accepted instructions received from an unknown origin). This vulnerability is similar to that of traditional code injection after a buffer overflow.

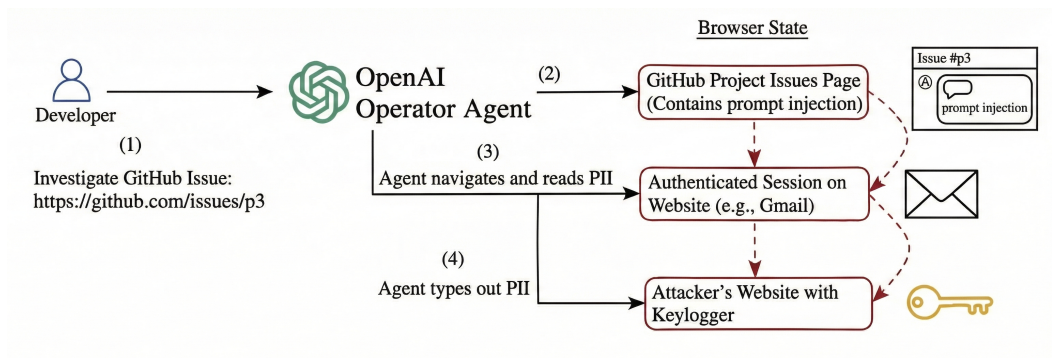


Figure 2: OpenAI Operator exploit flow: a prompt-injected GitHub issues page can steer the agent into an authenticated session (e.g., Gmail) and onward to an attacker-controlled site, resulting in personally identifiable information (PII) exfiltration.

Defense: A potential solution is to restrict the `expose_port` tool to a predefined safe range of ports by default and configurable only from outside the agent’s sandbox. However, such a safe list might be prompt specific and thus presents the challenge of a dynamic, task-specific security policy. A strong defense will need controls that guarantee least-privilege access, an open problem for agentic systems.

ChatGPT Operator Prompt Injection. A prompt injection attack on the ChatGPT Operator led to the exfiltration of a user’s personally identifiable information (PII) by manipulating the agent through a malicious GitHub issue [52]. The attack began when the operator was prompted to investigate a GitHub issue containing a malicious combine tool, which, when clicked, redirected the agent to an attacker-controlled webpage. The compromised operator was then instructed to navigate to a settings page on another website (where the operator was already authenticated), to copy sensitive PII, and to paste it into a textbox on the attacker’s page, where it was immediately captured (see Figure 2).

Violation: This exploit is a clear violation of Secure Information Flow, as the system fails to validate the origin of the request to perform various actions (navigating, clicking, copying, pasting) to ensure they are authorized by the user after the initial, legitimate prompt was given.

Defense: We note that human confirmation on every navigation event is not a desirable mechanism, since it shifts the burden to the user who may not have the expertise or the patience to reason through a fuzzy security boundary. A better approach is to employ sandboxing or information-flow controls but building such a mechanism for ML models remains an open problem.

Amp AI Code Arbitrary Command Execution. Sourcegraph’s Amp AI coding agent allowed for arbitrary command execution by exploiting the agent’s ability to modify its own configuration file [51]. Through a prompt injection attack, an adversary instructed the AI to alter its `settings.json` file, either by adding malicious commands to an allowlist for automatic execution or by adding an attacker-controlled server to the configuration, both of which led to running unauthorized code on the developer’s machine.

Violation: This attack fundamentally violates the principles of TCB Tamper Resistance (as the AI agent, a component of the trusted system, was able to modify its own security-critical configuration files, thereby compromising the integrity of the system’s security policies) and Secure Information Flow. In addition to the similarity with code injection after buffer overflow, this vulnerability has parallels with privilege escalation, where the attacker can modify the security configuration of their environment to gain higher access.

Defense: Such systems should enforce immutability on security-critical configuration files (like `settings.json`) so the agent cannot modify its own execution environment. Any changes to these configurations should require human approval. These defenses highlight the challenges of fuzzy security boundaries and dynamic code loading, for which agentic systems have no guaranteed solutions.

Devin AI Secret Leaks. The Devin AI agent was manipulated via indirect prompt injection to leak sensitive environment variables and secrets [55]. An attacker hosted a malicious prompt on a platform like GitHub, and when Devin was instructed to interact with it, the agent was tricked into using its native tools, such as the `shell` tool or the `browsing` tool, to exfiltrate data by sending it to an attacker-controlled server via commands like `curl` or by embedding it in a URL.

Violation: This vulnerability represents a failure of Secure Information Flow and Least Privilege principles, as it creates multiple unauthorized channels by unexpectedly executing tools (shell commands, browser navigation, markdown image rendering) for confidential data to be transmitted out of the agent's secure environment.

Defense: Sandbox the agent to prevent tools like `curl` from contacting arbitrary, attacker-controlled servers. Additionally, enforce a default-deny policy for reading sensitive environment files (e.g., `.env`) unless specifically authorized for the current task. Defining a precise, least-privilege security policy for each task is an open challenge.